

Trellix Deployment Guide

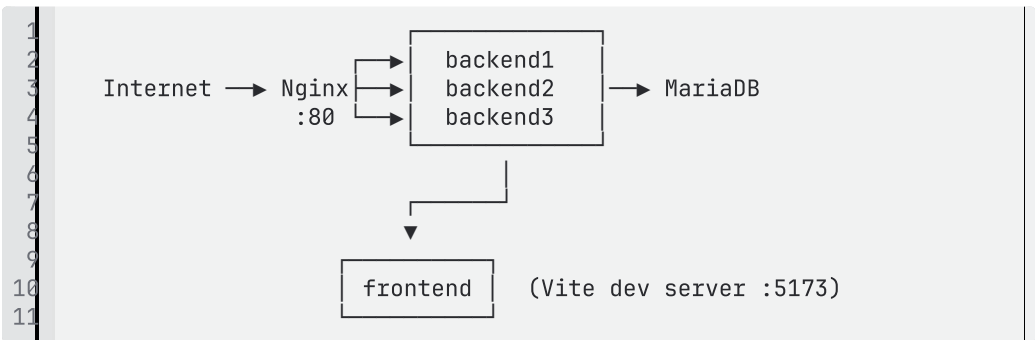
Author: Issa El Mokadem

Program: 2º DAW – I.E.S. Abdera

Project Type: Full-Stack Web Application (Task Board)

1. Architecture Overview

Trellix is deployed as a multi-container application orchestrated by Docker Compose. All services run in isolated containers communicating over a shared Docker network. Nginx acts as a reverse proxy, routing traffic to the backend API and frontend dev server.



1.1 Container Inventory

Service	Image	Port (Host)	Port (Container)	Purpose
nginx	nginx:alpine	80	80	Reverse proxy + static serving
backend1	python:3.12-slim	—	8000	FastAPI (replica 1)
backend2	python:3.12-slim	—	8000	FastAPI (replica 2)
backend3	python:3.12-slim	—	8000	FastAPI (replica 3)
db-init	python:3.12-slim	—	—	One-shot: creates DB schema, then exits

mariadb	mariadb:10.5	3307	3306	Database
frontend	node:20-slim	—	5173	Vite dev server with HMR

2. Service Configuration

2.1 Nginx — Reverse Proxy

Nginx is the single entry point. All traffic enters through port 80.

Location `/api/`: routes to the backend upstream, load-balanced across 3 replicas using `least_conn` algorithm.

```

1 upstream backend_api {
2     least_conn;
3     server backend1:8000 max_fails=3 fail_timeout=30s;
4     server backend2:8000 max_fails=3 fail_timeout=30s;
5     server backend3:8000 max_fails=3 fail_timeout=30s;
6 }
7
8 location /api/ {
9     proxy_pass http://backend_api;
10    proxy_set_header Host $host;
11    proxy_set_header X-Real-IP $remote_addr;
12    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
13    proxy_set_header X-Forwarded-Proto $scheme;
14 }
```

Location `/`: routes to the frontend Vite dev server with WebSocket support for Hot Module Replacement (HMR).

```

1 location / {
2     proxy_pass http://frontend:5173;
3     proxy_http_version 1.1;
4     proxy_set_header Upgrade $http_upgrade;
5     proxy_set_header Connection "upgrade";
6     proxy_set_header Host $host;
7 }
```

Health checks: `max_fails=3 fail_timeout=30s` — if a backend fails 3 health checks, it is removed from rotation for 30 seconds.

2.2 Backend — FastAPI

Built from `backend/Dockerfile`:

```

1 FROM python:3.12-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY . .
6 CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Development mode uses `--reload` for hot reloading (set in docker-compose command override).

Environment variables (injected by docker-compose):

Variable	Value	Purpose
DB_HOST	mariadb	Docker service name
DB_PORT	3306	Internal MariaDB port
DB_USER	trellix	Database user
DB_PASSWORD	trellix	Database password
DB_NAME	trellix	Database name

The backend configuration uses `docker-compose.yml` YAML anchors (`&backend-config` / `*backend-config`) to share the same build context, environment, volumes, and dependencies across all three replicas and the db-init service.

2.3 Database — MariaDB

```
1 mariadb:
2   image: mariadb:10.5
3   ports:
4     - "3307:3306"      # Exposed for external tools (Sequel Ace,
DBeaver)
5   environment:
6     - MYSQL_ROOT_PASSWORD=root
7     - MYSQL_DATABASE=trellix
8     - MYSQL_USER=trellix
9     - MYSQL_PASSWORD=trellix
10  volumes:
11    - mariadb_data:/var/lib/mysql # Persistent storage
```

Named volume `mariadb_data` : ensures data survives container restarts and rebuilds.

2.4 Database Initialization — db-init

A one-shot service that runs before the backends start:

```
1 db-init:
2   <<: *backend-config
3   command: python -m app.db_init
4   depends_on:
5     - mariadb
```

Execution order: `mariadb` starts → `db-init` creates schema and exits → `backend1/2/3` start only after `db-init` completes (`service_completed_successfully`).

2.5 Frontend — Vite Dev Server

```
1 FROM node:20-slim
2 WORKDIR /app
3 COPY package*.json ./
4 RUN npm install
5 COPY . .
6 EXPOSE 5173
7 CMD ["npm", "run", "dev", "--", "--host"]
```

The `--host` flag exposes the dev server to the Docker network so Nginx can proxy to it.

3. Environment Variables Summary

Backend (`docker-compose.yml`)

Variable	Value
DB_HOST	mariadb
DB_PORT	3306
DB_USER	trellix
DB_PASSWORD	trellix
DB_NAME	trellix

MariaDB

Variable	Value
MYSQL_ROOT_PASSWORD	root
MYSQL_DATABASE	trellix
MYSQL_USER	trellix
MYSQL_PASSWORD	trellix

Frontend

Variable	Value
VITE_API_URL	<http://localhost/api/v1 >

Note: The frontend accesses the API through Nginx, not directly to the backend. This is why the URL uses `localhost` (the browser's perspective) rather than a Docker service name.

4. Deployment Commands

4.1 First-time Setup

```
1 # Clone the repository
2 git clone https://github.com/issx03/daw-final-proyect-ielm.git
3 cd daw-final-proyect-ielm
4
5 # Build and start all services (detached mode)
6 docker compose up -d --build
7
8 # Verify all services are running (7 containers expected)
9 docker compose ps
```

Expected output: 7 containers: `nginx`, `backend1`, `backend2`, `backend3`, `frontend`, `mariadb`, plus `db-init` (exited successfully).

4.2 Starting and Stopping

```
1 # Start all services
2 docker compose up -d
3
4 # Stop all services (preserves volumes/data)
5 docker compose down
6
7 # Stop and remove volumes (fresh start, destroys data)
8 docker compose down -v
```

4.3 Viewing Logs

```
1 # All services
2 docker compose logs -f
3
4 # Specific service
5 docker compose logs -f backend1
6 docker compose logs -f nginx
```

4.4 Rebuilding After Changes

```
1 # Code changes in ./backend (mounted as volume): auto-reload via --
  reload
2 # No restart needed
3
4 # Dockerfile or dependency changes:
5 docker compose up -d --build
6
7 # Nginx config changes:
8 docker compose restart nginx
```

4.5 Running Backend Tests

```
1 docker compose exec -e PYTHONPATH=/app backend1 pytest tests/ -v
```

4.6 Accessing the Database

```
1 # Via docker exec
2 docker compose exec mariadb mysql -u trellix -ptrellix trellix
3
4 # Via external client (exposed on port 3307)
5 # Host: localhost, Port: 3307, User: trellix, Password: trellix, DB:
  trellix
```

5. Service Dependencies

The startup order is enforced by `depends_on` :

```
1 mariadb
2   └─ db-init (waits for mariadb, runs once, exits)
3       └─ backend1, backend2, backend3 (wait for db-init success)
4           └─ frontend (waits for all backends)
5               └─ nginx (waits for frontend + all backends)
```

6. Network Architecture

All services communicate over the default Docker Compose network (`<project>_default`).

Communication	Via	Protocol
Browser → Trellix	<code><http://localhost:80 ></code>	HTTP
Nginx → Backend	<code><http://backend1:8000 ></code> (internal)	HTTP
Backend → MariaDB	<code>mariadb:3306</code> (internal)	MySQL
Nginx → Frontend	<code><http://frontend:5173 ></code> (internal, with WebSocket)	HTTP/WS

No service is directly exposed to the host except:

- Nginx (port 80)
- MariaDB (port 3307 — for development/diagnostic tools only)

7. Load Balancing

Nginx distributes API requests across 3 backend replicas using the `least_conn` algorithm:

- **least_conn**: sends each new request to the backend with the fewest active connections
- **Health checks**: `max_fails=3 fail_timeout=30s` — 3 consecutive failures remove the backend from rotation for 30 seconds

- **Session affinity:** not configured (stateless JWT auth — any backend can handle any request)

7.1 Scaling

To add or remove backend replicas:

1. Copy/paste a backend service block in `docker-compose.yml` (backend4, backend5...)
2. Add the new server to the `upstream backend_api` block in `nginx.conf`
3. Run `docker compose up -d && docker compose restart nginx`

8. Data Persistence

Data	Storage	Survives
Database (tables, users, boards, etc.)	Named volume <code>mariadb_data</code>	<code>docker compose down</code>  , <code>docker compose down -v</code> 
Backend source code	Bind mount <code>./backend:/app</code>	Always (local filesystem)
Frontend source code	Bind mount <code>./frontend:/app</code>	Always (local filesystem)

9. Troubleshooting

9.1 Common Issues

Symptom	Probable Cause	Fix
<code>502 Bad Gateway</code> from Nginx	Backends not ready	Wait ~10s after <code>docker compose up</code> or check <code>docker compose logs backend1</code>
Backend can't connect to MariaDB	MariaDB not ready or wrong credentials	<code>docker compose logs mariadb</code> — check for "ready for connections"
Frontend shows blank page	Vite not started or port mismatch	<code>docker compose logs frontend</code>

"Network error" on API calls	VITE_API_URL misconfigured	Should be <http://localhost/api/v 1 > (browser → Nginx → backend)
Port 80 already in use	Another service on port 80	sudo lsof -i :80 → stop conflicting service or change nginx port

9.2 Port Conflict Resolution

If port 80 is unavailable, change in `docker-compose.yml`:

```
1 nginx:
2   ports:
3     - "8080:80"    # Map host port 8080 instead
```

9.3 Reset Everything

```
1 # Full reset: removes containers, volumes, networks
2 docker compose down -v
3
4 # Rebuild from scratch
5 docker compose up -d --build
```

10. Security Considerations

Concern	Mitigation
Database credentials in docker-compose.yml	Use <code>.env</code> file for production; current setup is for development
Direct database access (port 3307)	Remove port mapping in production or restrict to localhost
JWT secret key	Stored as environment variable, not hardcoded
CORS	Backend validates origin against allowed list
Nginx exposes only port 80	No backend or database ports exposed to the host
Frontend API URL	Proxied through Nginx — browser never talks directly to backend